

Courses

GET	/api/Courses	⌵
POST	/api/Courses	⌵
GET	/api/Courses/{id}	⌵
PUT	/api/Courses/{id}	⌵
DELETE	/api/Courses/{id}	⌵

Schemas

Course >

GET

GET /api/Courses

Try it out

Parameters

No parameters

Responses

Code	Description	Links
200	OK Media type text/plain Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "teacher": "string", "credits": 0}</pre>	No links

POST

POST /api/Courses

Try it out

Parameters

No parameters

Request body

application/json

Example Value | Schema

```
{  "id": 0,  "name": "string",  "teacher": "string",  "credits": 0}
```

Responses

Code	Description	Links
200	OK Media type text/plain Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "teacher": "string", "credits": 0}</pre>	No links

GET id

GET

/api/Courses/{id}

^

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	2
(path)	

Execute

Clear

Responses

Curl

curl -X 'GET' \ 'https://localhost:7120/api/Courses/2' \ -H 'accept: text/plain'

Request URL

https://localhost:7120/api/Courses/2

Server response

Code	Details
200	Response body { "id": 2, "name": "Databases", "teacher": "Anna Petrova", "credits": 4 } Download Response headers content-type: application/json; charset=utf-8 date: Thu, 10 Sep 2026 17:40:58 GMT server: Kestrel

PUT

PUT

/api/Courses/{id}

^

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	2
(path)	

Request body

application/json

{
 "id": 0,
 "name": "string",
 "teacher": "string",
 "credits": 0
}

Execute

Clear

Responses

Curl

curl -X 'PUT' \ 'https://localhost:7120/api/Courses/2' \ -H 'accept: */*' \ -H 'Content-Type: application/json' \ -d '{
 "id": 0,
 "name": "string",
 "teacher": "string",
 "credits": 0
}'

Request URL

https://localhost:7120/api/Courses/2

DELETE

DELETE

/api/Courses/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	2
(path)	

Execute

Clear

Responses

Curl

curl -X 'DELETE' \ 'https://localhost:7120/api/Courses/2' \ -H 'accept: */*'

Request URL

https://localhost:7120/api/Courses/2

Server response

Code	Details
204	Response headers
Undocumented	date: Thu, 10 Sep 2026 17:43:24 GMT server: Kestrel

Responses

Code	Description	Links
200	OK	No links

Почему нет id 100

Пустой объект это валидный ответ 200 OK, и клиент не может отличить «курса нет» от «курс существует, но все его поля случайно пустые». Код 404 Not Found это часть семантики протокола HTTP: он однозначно и машинно-читаемо говорит клиенту, что ресурс с данным идентификатором отсутствует

The screenshot shows a REST client interface with the following sections:

- DELETE /api/Courses/{id}**: The method and endpoint.
- Parameters**: A table with columns 'Name' and 'Description'. It contains one parameter:

Name	Description
id * required	integer(\$int32) (path)
- id * required**: A text input field containing the value '100'.
- Execute** and **Clear** buttons.
- Responses**:
 - Curl**:

```
curl -X 'DELETE' \
'https://localhost:7120/api/Courses/100' \
-H 'accept: */*'
```
 - Request URL**: `https://localhost:7120/api/Courses/100`
 - Server response**:
 - Code**: 404
 - Details**: Error: response status is 404
 - Response body**:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title": "Not Found",
  "status": 404,
  "traceId": "00-a1e22e50f6743f710d1c9a15cd69569f-7cab097d359dac5f-00"
}
```
 - Response headers**:

```
content-type: application/problem+json; charset=utf-8
date: Thu, 10 Sep 2026 17:44:03 GMT
server: Kestrel
```
- Responses**: A table with columns 'Code' and 'Description'. It contains one response:

Code	Description
200	OK
- Links**: No links

1.5 Контрольные вопросы

Что такое Web API?

Программный интерфейс, который позволяет разным приложениям обмениваться данными через HTTP, обычно возвращая данные в формате JSON или XML.

Чем Web API отличается от обычной веб-страницы?

Веб-страница возвращает HTML для отображения человеку в браузере, а Web API возвращает структурированные данные (JSON), предназначенные для обработки программой.

Что такое клиент-серверная архитектура?

Модель взаимодействия, где клиент отправляет запросы, а сервер их обрабатывает и возвращает ответы; клиент и сервер — это отдельные, независимо работающие компоненты.

Что такое HTTP request?

Сообщение, которое клиент отправляет серверу; содержит метод (GET, POST и т.д.), URL, заголовки и, при необходимости, тело запроса.

Что такое HTTP response?

Сообщение, которое сервер отправляет клиенту в ответ на запрос; содержит статус-код, заголовки и, при необходимости, тело ответа.

Что такое endpoint?

Конкретный URL-адрес API, привязанный к определённой операции (например, GET /api/courses/1).

Для чего используется HTTP GET?

Для получения (чтения) данных с сервера без изменения их состояния.

Для чего используется POST?

Для создания нового ресурса на сервере, передавая данные в теле запроса.

В чём отличие PUT от POST?

POST создаёт новый ресурс, а PUT обновляет (полностью заменяет) уже существующий ресурс по известному идентификатору; PUT также идемпотентен — повторный вызов с теми же данными не меняет результат.

Для чего используется DELETE?

Для удаления существующего ресурса на сервере по его идентификатору.

Что означает HTTP Status Code 200?

OK — запрос успешно обработан, и сервер вернул запрошенные данные.

Что означает 201?

Created — новый ресурс был успешно создан на сервере.

Что означает 204?

No Content — запрос выполнен успешно, но в ответе нет тела (часто используется для PUT/DELETE).

Что означает 404?

Not Found — запрошенный ресурс с указанным идентификатором не найден на сервере.

Что такое JSON?

Текстовый формат обмена данными (JavaScript Object Notation), основанный на парах «ключ-значение» и легко читаемый как человеком, так и программой.

Что делает атрибут [HttpGet]?

Помечает метод контроллера как обработчик HTTP GET-запросов на соответствующий маршрут.

Для чего используется [Route]?

Определяет шаблон URL, по которому доступен контроллер или конкретное действие (например, api/[controller]).

Что делает ControllerBase?

Базовый класс для Web API контроллеров без поддержки представлений (Views); предоставляет методы вроде Ok(), NotFound(), CreatedAtAction() для формирования HTTP-ответов.

Что такое Swagger?

Инструмент (на основе спецификации OpenAPI), который автоматически генерирует интерактивную документацию API и позволяет тестировать endpoints прямо из браузера.

Почему Web API удобно использовать для взаимодействия нескольких информационных систем?

Потому что API предоставляет единый, стандартизированный и не зависящий от языка программирования способ обмена данными (через HTTP и JSON), что позволяет разным системам, написанным на разных технологиях, легко интегрироваться друг с другом.